UniGuide

AI-Powered Personalised Academic Planning Agent

Production Architecture · Tech Stack · Phase Plan

Kaggle · 5-Day AI Agents Intensive · Vibe Coding Course with Google

Built with Google ADK + Gemini + ChromaDB + Google Calendar API

1. Required Tech Skills

Core AI / Agent skills

Skill	Depth needed	Used for
Google ADK (Agent Development Kit)	**Primary**	Orchestrator + all agent definitions, tool registration, runner
Gemini API (google-generativeai)	**Primary**	LLM backbone for all agents, embedding generation
Prompt engineering	Intermediate	System prompts per agent, structured output instructions
Multi-agent orchestration	Intermediate	Sequential / parallel agent pipelines, state hand-off
RAG (Retrieval-Augmented Generation)	Intermediate	Course catalog semantic search via ChromaDB
Tool / function calling	Intermediate	ADK tool wrappers for PDF, Calendar, DB
Pydantic structured outputs	Intermediate	Type-safe schemas between agents
Embedding models	Basic	text-embedding-004 for course vectorisation

Python packages — full list

Google ADK & LLM

Package	Version	Purpose
google-adk	latest	Agent orchestration framework — primary
google-generativeai	>=0.7	Gemini 1.5 Flash / Pro API client
google-auth	>=2.0	OAuth2 for Calendar + ADK auth
google-auth-oauthlib	latest	OAuth2 flow helpers
google-api-python-client	latest	Google Calendar REST API

Vector store & embeddings

Package	Version	Purpose
chromadb	>=0.5	Local persistent vector store for course catalog
sentence-transformers	>=3.0	Fallback local embeddings (all-MiniLM-L6-v2)
numpy	>=1.26	Embedding arithmetic, similarity helpers

Data & persistence

Package	Version	Purpose
sqlalchemy	>=2.0	ORM for StudentProfile + SemesterPlan (SQLite)
alembic	latest	DB migrations
pydantic	>=2.0	Structured agent I/O schemas, config validation
pydantic-settings	latest	Environment-based config (.env support)
pandas	>=2.0	Course catalog CSV ingestion, grade table parsing

Document & file handling

Package	Version	Purpose
pdfplumber	>=0.11	Extract tables + text from student transcripts
PyMuPDF (fitz)	>=1.24	Fallback PDF handling, image extraction
openpyxl	latest	Read Excel exports from campus portal
python-docx	latest	Read/write Word documents if needed

API, CLI & utilities

Package	Version	Purpose
click	>=8.0	CLI interface (entry point for demo)
rich	latest	Terminal output formatting, progress bars, tables
python-dotenv	latest	Load .env credentials (API keys, OAuth)
httpx	latest	Async HTTP client for any external API calls
tenacity	latest	Retry logic for LLM + Calendar API calls
loguru	latest	Structured logging across agents

Testing & dev

Package	Version	Purpose
pytest	>=8.0	Unit + integration tests
pytest-asyncio	latest	Async agent test support
pytest-mock	latest	Mock LLM calls and Calendar API in CI
ruff	latest	Linting + formatting (fast, replaces flake8/black)

2. Production-Level Architecture

Guiding principles

- **Separation of concerns:** each agent owns exactly one domain — parsing, retrieval, scheduling
- **Structured hand-offs:** Pydantic schemas enforce contracts between agents, no free-form string passing
- **Tool isolation:** every external call (PDF, DB, Calendar, ChromaDB) is a registered ADK tool with retry + error handling
- **Stateless agents:** all state lives in SQLite / ChromaDB; agents are re-entrant and testable in isolation
- **Graceful degradation:** if Calendar push fails, plan is still saved locally; if PDF parse fails, manual input mode activates

Layer breakdown

Layer	Component	Technology	Responsibility
Entry	CLI	Click + Rich	Parse args, validate inputs, invoke runner
Entry	Session manager	Python / SQLite	Create/resume student session, load profile
Orchestration	Orchestrator agent	Google ADK	Route tasks, merge outputs, produce final report
Agent	Profile Analyst	Google ADK + Gemini	Parse transcript, compute credit gap, flag weak areas
Agent	Course Recommender	Google ADK + Gemini	RAG query, prereq filter, rank courses
Agent	Scheduler	Google ADK + Gemini	Distribute courses, balance workload, push Calendar
Tool	pdf_parse_tool	pdfplumber / PyMuPDF	Extract grade tables from transcript PDF
Tool	credit_gap_tool	Python / SQLAlchemy	Query DB for completed credits, compute gap
Tool	rag_search_tool	ChromaDB + Gemini embeddings	Semantic course retrieval with metadata filters
Tool	db_read_tool	SQLAlchemy	Read StudentProfile, course catalog
Tool	db_write_tool	SQLAlchemy	Write SemesterPlan back to SQLite
Tool	calendar_push_tool	Google Calendar API	Create course + exam events, return event IDs
Store	Student DB	SQLite (SQLAlchemy)	StudentProfile, SemesterPlan, grades

Store	Course vector store	ChromaDB (persistent)	Course embeddings + metadata
Store	Agent memory	JSON / SQLite	Multi-turn session state
Output	Report renderer	Rich / Markdown	Formatted semester plan to terminal + .md file

Folder structure

```
uniguide/
├── agents/
│   ├── orchestrator.py      # Root ADK agent — coordinates all specialists
│   ├── profile_analyst.py   # Transcript parsing + credit gap agent
│   ├── course_recommender.py # RAG search + ranking agent
│   └── scheduler.py         # Semester distribution + Calendar agent
├── tools/
│   ├── pdf_parse_tool.py    # pdfplumber wrapper
│   ├── rag_search_tool.py   # ChromaDB query wrapper
│   ├── credit_gap_tool.py   # DB credit computation
│   ├── db_tool.py           # SQLAlchemy read/write
│   └── calendar_tool.py     # Google Calendar API wrapper
├── models/
│   ├── student.py           # StudentProfile, Grade Pydantic models
│   ├── course.py            # Course, CourseRecommendation models
│   └── plan.py              # SemesterPlan, SemesterBlock models
├── db/
│   ├── database.py          # SQLAlchemy engine, session factory
│   └── migrations/          # Alembic migration scripts
├── ingestion/
│   ├── catalog_ingestor.py  # CSV → ChromaDB pipeline
│   └── transcript_parser.py # Standalone PDF → JSON
├── config.py                # pydantic-settings env config
├── cli.py                   # Click entry point
├── runner.py                # ADK runner, session bootstrap
├── data/
│   ├── sample_catalog.csv   # 80–100 courses for demo
│   └── sample_profile.json  # Demo student profile
├── tests/
│   ├── test_tools.py
│   ├── test_agents.py
│   └── conftest.py
├── .env.example
├── requirements.txt
└── README.md
```

Agent contracts (Pydantic schemas)

```
# Profile Analyst output
class ProfileAnalysis(BaseModel):
    credits_completed: int
```

```
    credits_remaining: int
    completed_course_ids: list[str]
    weak_subjects: list[str]          # tags where grade < threshold
    strong_subjects: list[str]
    gpa: float | None

# Course Recommender output
class CourseRecommendation(BaseModel):
    course_id: str
    title: str
    credits: int
    relevance_score: float            # 0-1, from RAG
    rationale: str                    # LLM-generated explanation
    semester_offered: list[int]
    prerequisites_met: bool

# Scheduler output
class SemesterBlock(BaseModel):
    semester_number: int
    courses: list[CourseRecommendation]
    total_credits: int
    calendar_event_ids: list[str]

class SemesterPlan(BaseModel):
    student_id: str
    semesters: list[SemesterBlock]
    total_planned_credits: int
    graduation_semester: int
```

Campus portal data — what to feed in

Since you can pull data from your TU Dortmund campus portal (LSF / BOSS system), here is exactly what to capture and how it maps to the system:

Campus portal info	Where to find it	How UniGuide uses it
Course catalog listing	LSF → Vorlesungsverzeichnis	Ingest as CSV into ChromaDB — title, description, ECTS, semester, prerequisites
Your transcript / grades	BOSS → Notenspiegel (PDF export)	pdf_parse_tool extracts course names, grades, ECTS completed
Module handbook	Program page PDF	Extracts mandatory vs elective split, prerequisite chains
Exam dates	LSF → Prüfungsplan	Scheduler agent creates exam reminder events in Calendar
Enrollment periods	LSF → Belegungsfristen	Stored in config; system warns if plan requires enrollment action

Credit requirements	Program Studienordnung PDF	Hardcoded per degree type: MSc Data Science TU Dortmund = 120 ECTS

For the demo you can export your own Notenspiegel PDF from BOSS and use the real TU Dortmund course catalog. This makes the demo far more compelling than synthetic data.

3. Phase-by-Phase Build Plan

Four phases. Phases 1–3 produce a working submission. Phase 4 is stretch / post-Kaggle.

Phase 1 — Foundation & Data Layer

Goal: project skeleton, databases wired, course catalog in ChromaDB, transcript parser working.

Task	File / component	Done when
Initialise repo + install all packages	requirements.txt, .env.example	pip install -r requirements.txt runs clean
Define all Pydantic models	models/student.py, course.py, plan.py	Models import without error, all fields typed
SQLAlchemy DB setup + Alembic migration	db/database.py, migrations/	uniguide.db created with correct schema
config.py with pydantic-settings	config.py	All secrets load from .env
Build catalog_ingestor.py	ingestion/catalog_ingestor.py	sample_catalog.csv ingested → ChromaDB collection populated
Build transcript_parser.py	ingestion/transcript_parser.py	Sample PDF → ProfileAnalysis JSON output
Unit test both ingestion scripts	tests/test_tools.py	pytest passes on happy path + edge cases

Campus portal input needed: Export your Notenspiegel PDF from BOSS. Export or screenshot the course list from LSF Vorlesungsverzeichnis. We can build the catalog CSV together from whatever columns LSF shows.

Phase 2 — Tools & Individual Agents

Goal: each ADK tool and agent works in isolation, tested individually.

Task	File / component	Done when
pdf_parse_tool — ADK tool wrapper	tools/pdf_parse_tool.py	Tool returns ProfileAnalysis for any TU Dortmund transcript
credit_gap_tool — compute remaining credits	tools/credit_gap_tool.py	Correct gap calculated vs program requirement
rag_search_tool — ChromaDB semantic query	tools/rag_search_tool.py	Returns top-5 courses for a sample interest query
db_tool — read/write SemesterPlan	tools/db_tool.py	Plan persisted and retrieved correctly

calendar_push_tool — gcal event creation	tools/calendar_tool.py	Dry-run prints events; live mode creates in test calendar
Profile Analyst agent	agents/profile_analyst.py	Agent calls pdf_parse_tool + credit_gap_tool, returns ProfileAnalysis
Course Recommender agent	agents/course_recommender.py	Agent calls rag_search_tool, returns ranked CourseRecommendation[]
Scheduler agent	agents/scheduler.py	Agent distributes courses, calls calendar_push_tool, returns SemesterPlan

Phase 3 — Orchestration, CLI & Demo Polish

Goal: end-to-end pipeline runs on a real student profile, Calendar populates, report renders, ready to record the video.

Task	File / component	Done when
Orchestrator agent — wires all three specialists	agents/orchestrator.py	Full pipeline runs: PDF in → SemesterPlan + Calendar events out
Runner bootstrap — session + ADK runner	runner.py	Single function call invokes orchestrator
CLI entry point	cli.py	python -m uniguide --profile p.json --transcript t.pdf works
Markdown report renderer	runner.py (post-process)	Clean .md file with semester table + course rationale written to disk
Rich terminal output	cli.py	Colour-coded summary, progress spinner during agent calls
Error handling + graceful degradation	All agents + tools	Calendar failure → plan still saved; PDF failure → prompts manual entry
README with setup steps	README.md	Reviewer can run from clone in under 10 minutes
Demo recording	YouTube (unlisted or public)	5-minute screen recording: input → agents → Calendar → report
Kaggle writeup	Kaggle submission page	2500-word writeup with architecture, decisions, demo link

Phase 4 — Stretch Goals (post-Kaggle)

Goal: productionise into a shareable tool for MSc students at TU Dortmund and beyond.

Feature	Tech	Value

Streamlit web UI	streamlit	Non-technical students can use it without CLI
Multi-university support	University config YAML	Parameterise credit systems (ECTS vs US), catalog sources
Grade trend analysis	pandas + Gemini	Spot trajectory: improving/declining across subjects
Peer anonymised comparison	Aggregated DB query	Show what students in similar positions typically took
Professor/course rating integration	Web scraping or RateMyProfessor API	Add social signal to recommendations
Automatic catalog sync	Scheduled job + LSF scraper	Keep ChromaDB fresh without manual CSV export

What to provide next

To start building immediately, share these from your campus portal:

#	What to share	Format	Used in
1	Your Notenspiegel (grade transcript)	PDF export from BOSS	Testing pdf_parse_tool on real data
2	Course list from LSF Vorlesungsverzeichnis	Screenshot or CSV export	Building sample_catalog.csv for ChromaDB
3	Your program's Studienordnung or module handbook	PDF from TU Dortmund program page	Extracting mandatory/elective split + prereqs
4	How many semesters remaining + your interests	Plain text	Seeding sample_profile.json for the demo

With items 1–3, we can build and test on your real data, which will make the Kaggle demo significantly more compelling than synthetic examples.